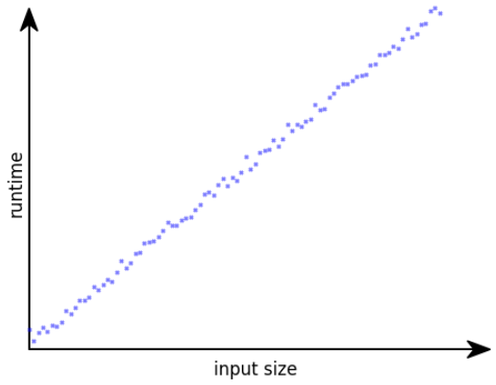
Runtime Analysis

Institute of
Algorithms and Theory

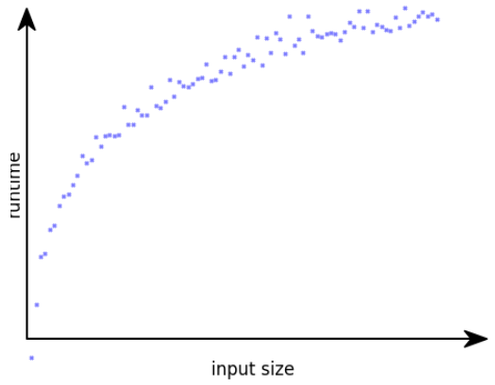
Yannic Maus

Runtime Measurements:

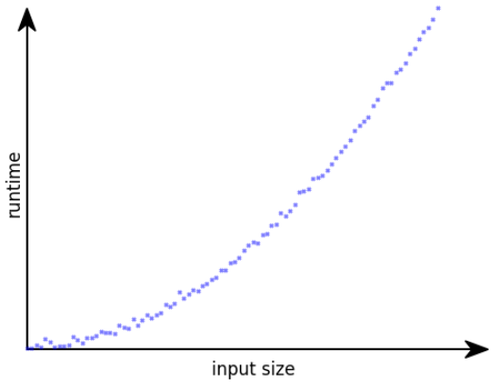
Measured runtimes from executing these algorithms.



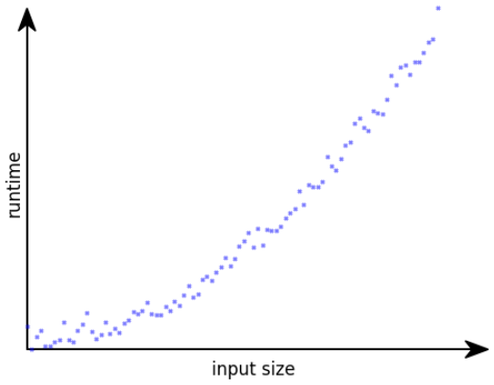
Linear Search



Binary Search



Selection Search



Insertion Search

Which one is asymptotically fastest?

How to express this mathematically?

Which model of a computer?

Church's Thesis:
The set of algorithmically solvable problems is the same for all reasonable models of computation.

Sequential Computability Hypothesis:
The required runtime for a problem on two different sequential computation models differs by at most a polynomial factor.

We will soon see what that means

This is not good enough for our purpose…

Runtime Analysis,

How can we analyze the runtime of an algorithm?

It's different on each computer...

Depends on the compiler, programming language, etc.

We need an abstract measure to assess runtime.

Idea: Count the number of (basic) operations

- Instead of measuring time directly
- Independent of computer, compiler

A good measure of runtime if all basic operations take roughly the same amount of time.

Basic Operations

What is a Basic Operation?

Simple arithmetic operations:

- +, -, *, /, % (mod), …

A memory access:

- Reading a variable, assigning a variable
- Is this really a basic operation?

A function call:

- Of course, just the jump to the function

Intuitively: One line of program code, **Better:** One line of machine code

Even better (?): One processor cycle

Key takeaway: *It's only important that the number of basic operations is roughly proportional to the runtime.*

Random Access Machine

Random Access Machine (RAM)

Standard model for analyzing algorithms.

Basic operations (like "defined") each take one time unit.

All memory accesses are equally expensive:

- Each memory cell (1 machine word) can be read or written in 1 time unit.
- Ignores memory hierarchies but is a reasonable assumption in most cases.

Rule of Thumb: For n memory cells, use $O(\log n)$ bits.

In practice many things matter as well, e.g., times for accessing different types of memory can make a huge difference. But that difference is ***only a constant-factor*** *and in this lecture we don't really care for constant factors. We will see why …*

Selection Sort: Runtime

```
SelectionSort(A):
for i = 0 to n - 2 do
    minIdx = i
    for j = i + 1 to n - 1 do
        if A[j] < A[minIdx] then
            minIdx = j
    swap(A[i], A[minIdx])
```

$\leq c_1$

$\leq c_2$

$\geq c_2'$

$\leq c_3$

T(n)=#BasicOperations ≤ c · #iterations of the inner for-loop=:x(n)

$$x(n) \leq \sum_{i=0}^{n-2} n - i = \sum_{h=2}^{n} h \leq \sum_{h=0}^{n} h = n \cdot \frac{n+1}{2} \leq n^2$$

$$\Rightarrow T(n) \leq c_{UB} \cdot x(n) = c_{UB} \cdot n^2$$

$$\Rightarrow T(n) \geq c_{LB} \cdot x(n) = c_{LB} \cdot n^2$$

Selection Sort: Upper and lower runtime bound

Lemma: There exists a constant $\boldsymbol{c_{UB} > 0}$, such that $\boldsymbol{T(n) \leq c_{UB} \cdot n^2}$.

Lemma: There exists a constant $\boldsymbol{c_{LB} > 0}$, such that $\boldsymbol{T(n) \geq c_{LB} \cdot n^2}$.

Summary

- We calculate a quantity that is proportional to the runtime.
- We don't want to calculate anything else:
  - The analysis should be independent of computer, compiler, etc.
  - We want statements that will still be valid in 10/100/... years.
- We will always have statements of the following form:

- *There exists a constant* C *s.t.* $T(n) \leq C \cdot f(n)$ holds.
- *There exists a constant* C *s.t.* $T(n) \geq C \cdot f(n)$ holds.

The function $f(n)$ is the asymptotically relevant part of the runtime!

*The constant C depends on the computer/compiler/etc, the function $f(n)$ does not!

Simplifications for runtime analysis

Simplification 1: The most important parameter is the size of the input n. Therefore, we consider the runtime $T(n)$ as a function of n.

- And we ignore other properties of the input.

(see previous lecture)

The number of basic operations can depend on various properties of the input.

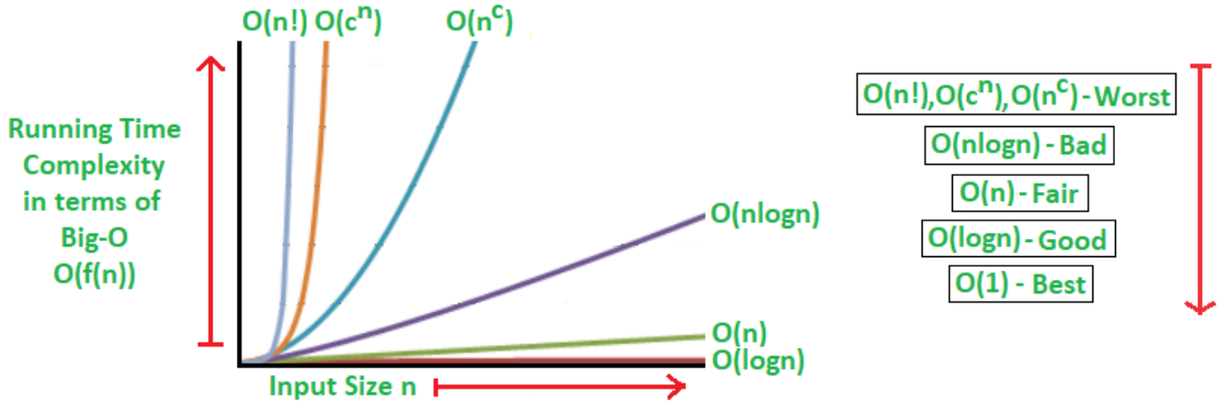
- Length of the input, for example, in sorting: random, pre-sorted, etc.

So far: The number of basic operations is proportional to the runtime.

We can achieve this without counting the exact number of basic operations!

Simplification 2: We calculate only an upper bound (or a lower bound) for the number of basic operations. *So that the bound is still proportional...*

Landau Notation



one of THE central topics of this class

Landau notation informal

There exists a constant C s.t. $T(n) \leq C \cdot f(n)$ holds.

Instead: $T(n) \in O(f(n))$ (speak: ”Oh” of $f(n)$)

There exists a constant C s.t. $T(n) \geq C \cdot g(n)$ holds.

Instead: $T(n) \in \Omega(g(n))$ (speak “Omega” of $g(n)$)

For Selection Sort:

$T(n) \in O(n^2)$, $T(n) \in \Omega(n^2)$, $T(n) \in \Theta(n^2)$

- $\Theta(n^2)$ sets of functions that grow like n^2
- $O(n^2)$ sets of functions that grow like n^2 or slower

(formal definitions on next slide)

Landau Notation: Definition

Big-O (O): $O(g) = \{ \mathrm{f} \mid$ there exist constants $c > 0$ and n_0 such that for all $n \geq n_0$,
$$|f(n)| \leq c \cdot |g(n)| \}$$
Interpretation: g is an upper bound on growth rate.

Big-Omega (Ω): $\Omega(g) = \{ \mathrm{f} \mid$ there exist constants $c > 0$ and n_0 such that for all $n \geq n_0$,
$$|f(n)| \geq c \cdot |g(n)| \}$$
Interpretation: g is a lower bound on growth rate.

Big-Theta (Θ): $\Theta(g) = \{ \mathrm{f} \mid$ there exist constants $c_1, c_2 > 0$ and n_0 such that for all $n \geq n_0$,
$$c_1 \cdot |g(n)| \leq |f(n)| \leq c_2 \cdot |g(n)| \}$$
Interpretation: g has asymptotically the same growth rate.

Formally, $O(g), \Omega(g), \Theta(g)$ are **sets** of functions, but one usually misuses notation and writes
$$f = O(g),\ f = \Omega(g), \text{ and } f = \Theta(g)$$

One-sided "equality". $f = O(g)$ does not automatically mean $g = O(f)$

Examples

Notation:

- $T(n) = O(n^2)$ instead of $T(n) \in O(n^2)$
- $f(n) = n^2 + O(n)$: $f(n) \in \{g(n) \mid \exists h(n) \in O(n) \text{ s.t. } g(n) = n^2 + h(n)\}$

Example:

- $f_1(n) = 200 \cdot n^2 + 5n + 3$,
- $f_2(n) = n^3$

Claim: $\boldsymbol{f_1(n) = O(f_2(n))}$

Pick $C = 2$ and $n_0 = 200$

Then for all $n \geq n_0$:

$$200 \cdot n^2 + 5n + 3 \leq n^3 + 5n + 3 \leq n^3 + n^3 = 2n^3$$

Examples

Sort these functions $f_{a_1}, f_{a_2}, \ldots$ such that $f_{a_i} = O(f_{a_{i+1}})$ *holds.*

- $f_3(n) = 100\, n^{333}$
- $f_4(n) = n^{0.01}$
- $\mathrm{f}_5(\mathrm{n}) = \log_2 \mathrm{n}$
- $f_6(n) = 0.01 \cdot \log_{10} n$
- $f_7(n) = \sqrt{n}$
- $f_8(n) = n/\log n$
- $f_9(n) = e^n$

Next week during lecture time:
Tutorial on Landau Notation

Landau Notation: Definition

Little-o (o): $o\big(g(n)\big) = \{\text{function f} \mid \text{for every } \varepsilon > 0\text{, there exists an } n_0 \text{ such that for all } n \geq n_0,$
$$|f(n)| < \varepsilon \cdot |g(n)|\}$$

Interpretation: Strict upper bound (grows strictly slower).

Little-omega (ω): $\omega(g(n)) = \{\text{f} \mid \text{for every } \varepsilon > 0\text{, there exists an } n_0 \text{ such that for all } n \geq n_0,$
$$|f(n)| > \varepsilon \cdot |g(n)|\}$$

Interpretation: Strict lower bound (grows strictly faster).

$$f \in o(g) \Rightarrow f \in O(g)$$
$$f \in \omega(g) \Rightarrow f \in \Omega(g)$$

Landau-Notation: Equivalent Characterization

Big-O(O): $f(n) = O(g(n))$ iff

$$\limsup_{n\to\infty} \frac{|f(n)|}{|g(n)|} < \infty$$

Big-Omega (Ω): $f(n) = \Omega(g(n))$ iff

$$\liminf_{n\to\infty} \frac{|f(n)|}{|g(n)|} > 0$$

Theta (Θ): $f(n) = \Theta(g(n))$ iff

$$0 < \liminf_{n\to\infty} \frac{|f(n)|}{|g(n)|} \leq \limsup_{n\to\infty} \frac{|f(n)|}{|g(n)|} < \infty$$

Little-o (o): $f(n) = o(g(n))$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|g(n)|} = 0$$

Little-omega (ω): $f(n) = \omega(g(n))$ iff

$$\lim_{n\to\infty} \frac{|f(n)|}{|g(n)|} = \infty$$

Insertion Sort

Recall:

Invariant: The beginning (prefix) of the array is sorted.

- Initially, only the first element is sorted, gradually more elements are added.
- Gradually insert each element in its correct position.

```
InsertionSort(array):
for i = 1 to n-1 do
    // Prefix A[0..i] is already sorted
    pos = i
    while (pos > 0) and (A[pos] < A[pos-1]) do
        swap(A[pos], A[pos-1])
        pos = pos - 1
```

Insertion Sort

```
InsertionSort(array):
for i = 1 to n-1 do
    // Prefix A[0..i] is already sorted
    pos = i
    while (pos > 0) and (A[pos] < A[pos-1]) do
        swap(A[pos], A[pos-1])
        pos = pos - 1
```

Runtime T(n)=O(#iterations of the while loop)=:x(n)

$$\mathrm{x(n)} \leq O\left(\sum_{i=1}^{n-1} i\right) = O\left(n \cdot \frac{n+1}{2}\right) = O(n^2)$$

$$\mathrm{x(n)} \geq \sum_{i=1}^{n-1} 1 = \mathrm{n} = \Omega(n)$$

How good is quadratic runtime?

Quadratic runtime: 2x larger input □ 4x larger runtime

this is often suboptimal in practice

- Pick an algorithm with $\mathrm{T(n)} \approx \mathrm{n}^2$ basic operations like insertion sort
- Assume 1Ghz CPU with 1 basic operation per CPU cycle

Input size n	**Input size Memory**	**Runtime**
10^3	$\approx 4KB$	$(10^{\mathbf{3}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=1ms
10^6	$\approx 4\ MB$	$(10^{\mathbf{6}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=16.7 minutes
10^9	$\approx 4\ GB$	$(10^{\mathbf{9}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=31.7 years

Too slow for large instances!

Is linear runtime much better?

- Pick an algorithm with $\mathbf{T(n)} \approx \mathbf{n}$ basic operations like insertion sort
- Assume 1Ghz CPU with 1 basic operation per CPU cycle

Input size n	**Input size Memory**	**Runtime**
10^3	$\approx 4KB$	$10^{\mathbf{3}} \cdot \mathbf{10^{\wedge} - 9}$ s=~~1ms~~ 10^-6s
10^6	$\approx 4\ MB$	$10^{\mathbf{6}} \cdot \mathbf{10^{\wedge} - 9}$ s=~~16.7 minutes~~ 10^-3s
10^9	$\approx 4\ GB$	$10^{\mathbf{9}} \cdot \mathbf{10^{\wedge} - 9}$ s=~~31.7 years~~ 1s

Much much better!